

**407 Proxy
Authentication
Required**



200 OK

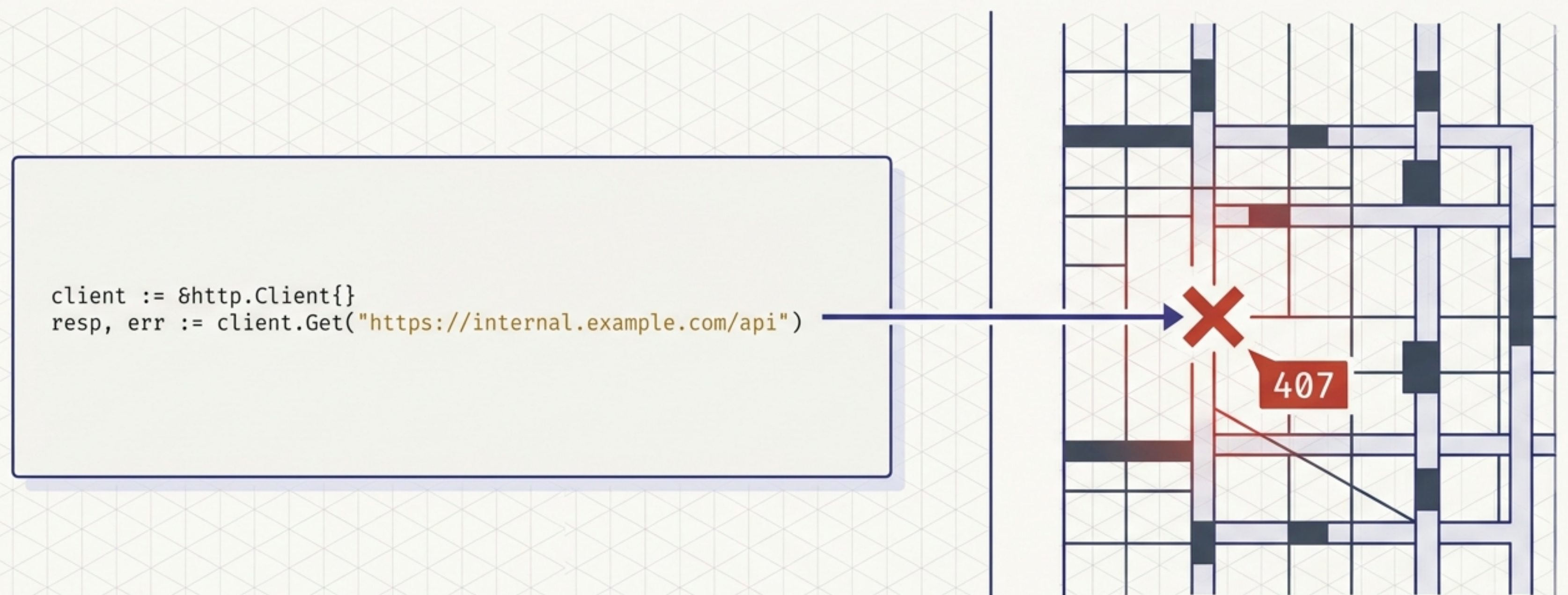
Transparent Corporate Proxy Authentication in Go

Solving the 407 Challenge with Native Windows SSO, NTLM, and Kerberos.

github.com/sanjaynagpal/proxyauth

The Enterprise Firewall Wall

```
client := &http.Client{}  
resp, err := client.Get("https://internal.example.com/api")
```



The diagram illustrates a network architecture where a Go application on the left attempts to connect to an internal API endpoint. A blue arrow represents the network request, which is blocked by a complex 'firewall wall' on the right. The wall is depicted as a grid of black and white squares with a red 'X' and a '407' status code indicating a failed authentication attempt. The background features a light blue grid pattern.

Building Go apps in enterprise environments often fails immediately because standard network libraries cannot natively authenticate against corporate proxies using NTLM or SPNEGO/Kerberos.

Why net/http Drops the Connection

Broken Pipeline



Basic Auth Only.
Go only knows how to
preload Proxy-
Authorization: Basic
from URL credentials.

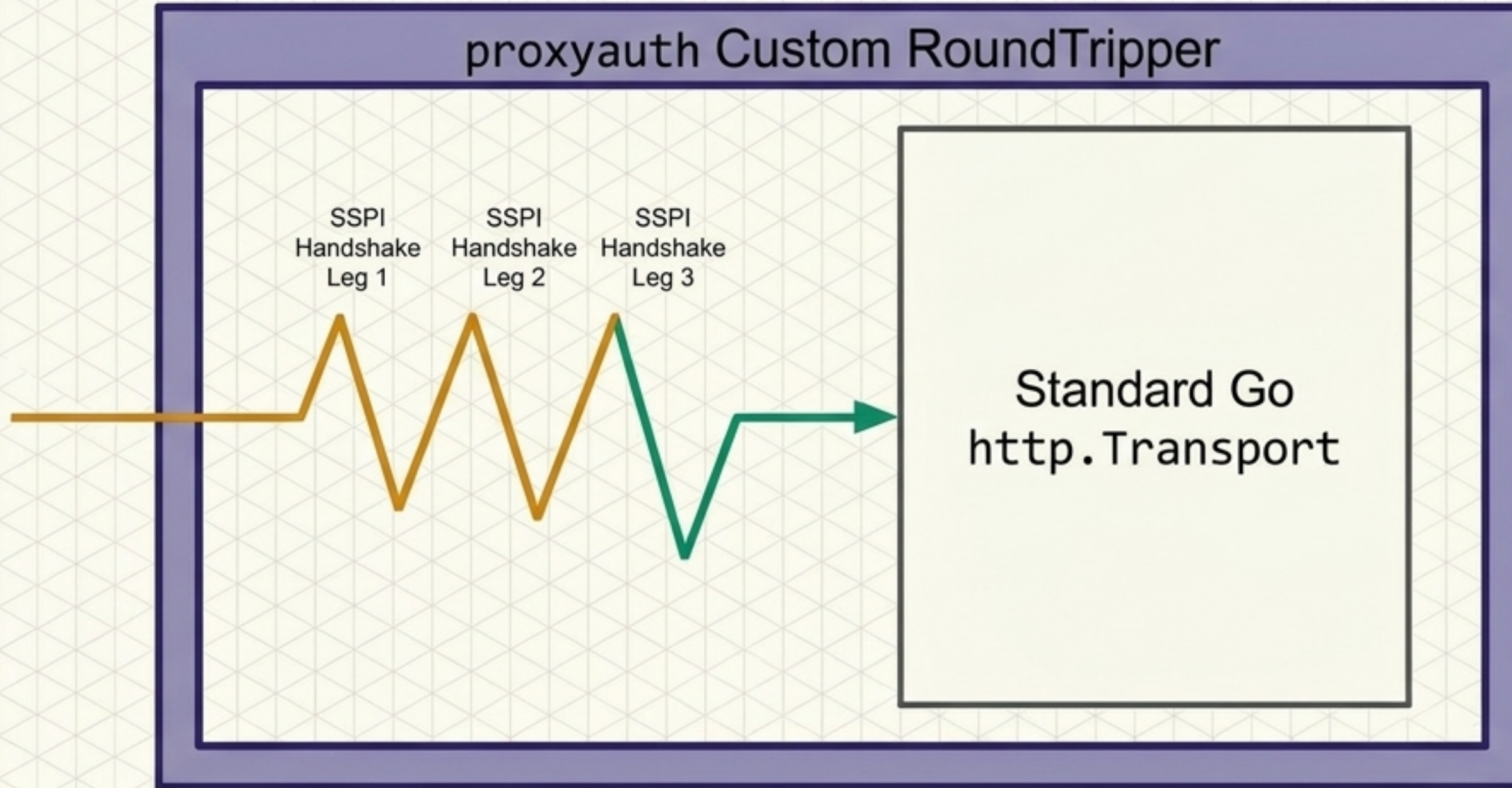
Connection Teardown.

For HTTPS requests, the 407 arrives
during the CONNECT phase.

http.Transport instantly tears down
the connection on a non-200 response.

Multi-leg protocols like NTLM authenticate the TCP connection itself. Standard Go surfaces the 407 as a terminal RoundTrip error, tearing down the TCP socket before the required multi-message back-and-forth can occur.

Taking Over the Tunnel



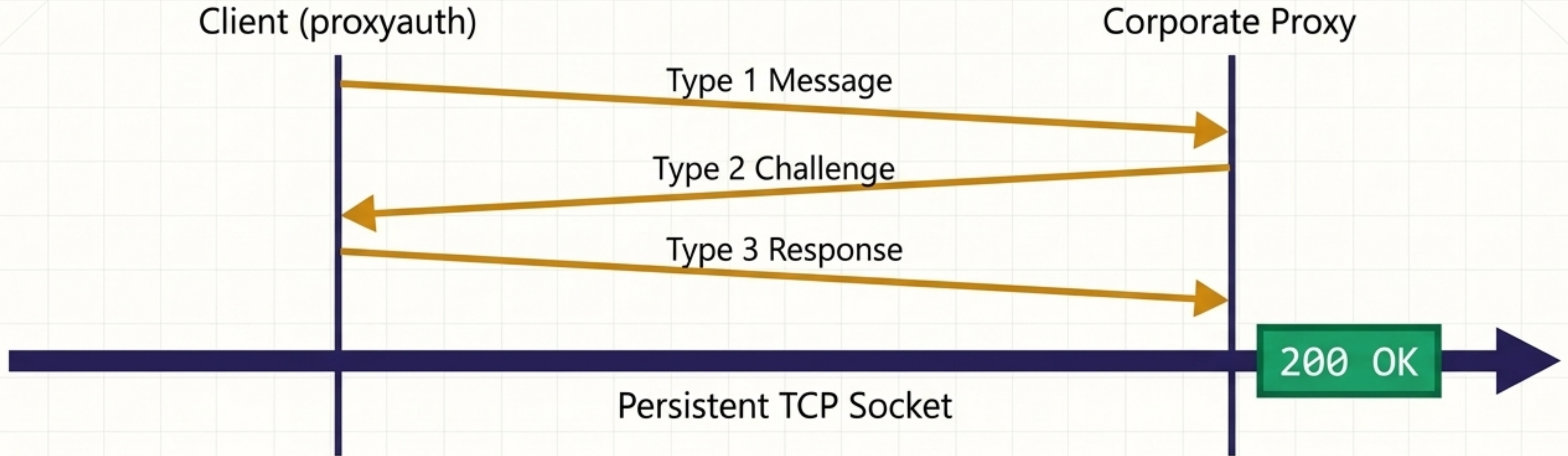
`proxyauth` dials the proxy, handles the multi-leg SSPI handshake on a single kept-alive connection, and hands the fully authenticated tunnel back to Go's standard library to perform the origin TLS handshake.

Diagnostic Comparison Matrix

Feature	net/http	proxyauth
Basic Auth	✓	✓
NTLM / Negotiate (SPNEGO/Kerberos)	✗	✓
HTTPS CONNECT Handling	✗	✓
Single-Connection Multi-Leg Auth	✗	✓
Zero-Config Windows SSO	✗	✓

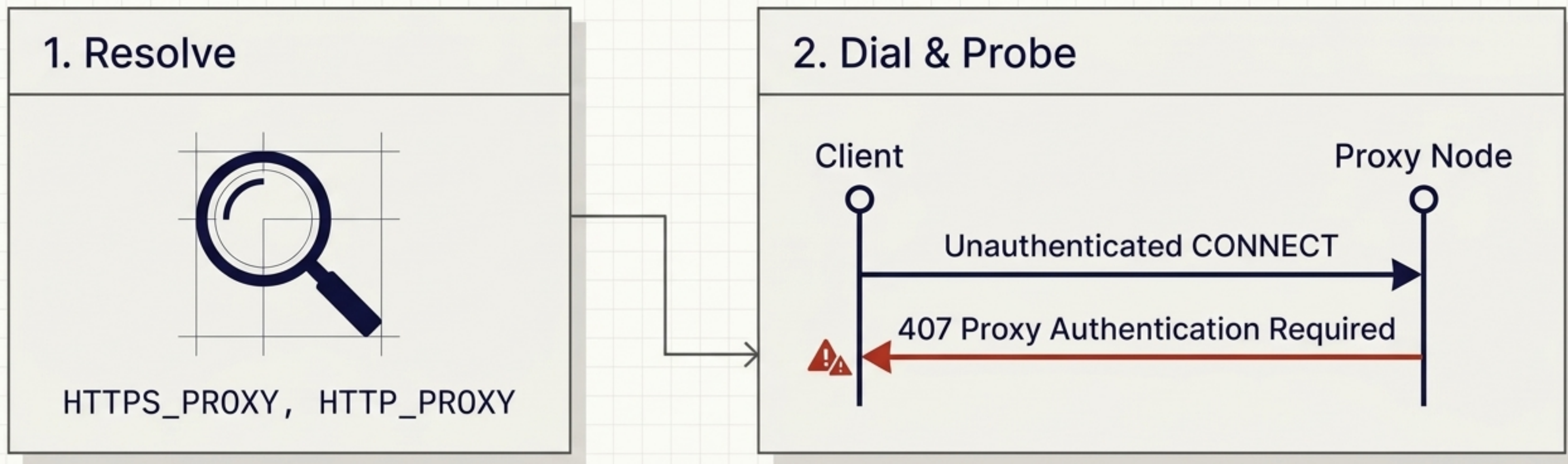
proxyauth bridges the gap between Go's primitive basic auth and enterprise-grade network security requirements.

The Single-Connection Imperative



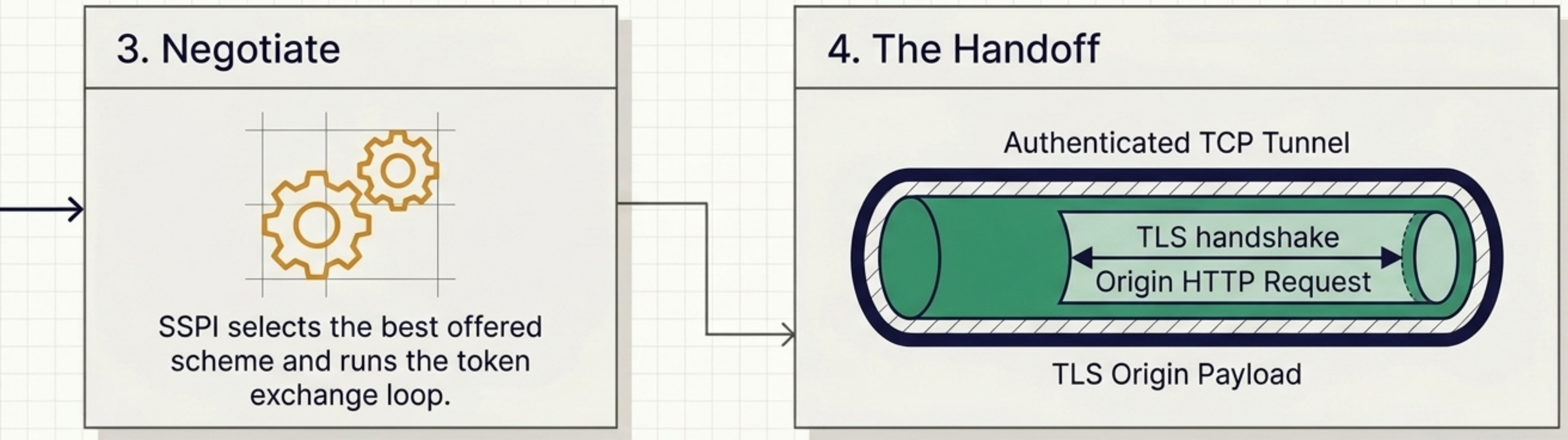
NTLM and Negotiate authenticate the TCP socket. proxyauth keeps the connection alive across the entire challenge-response loop, feeding Proxy-Authenticate tokens back into Windows SSPI until successful.

Anatomy of a Request: The Intercept



The package intentionally triggers the 407 error on the initial unauthenticated request to force the proxy to advertise its supported authentication schemes.

Anatomy of a Request: Negotiation & Handoff



Once the proxy yields a 200 OK, proxyauth steps back. The authenticated tunnel is handed over to standard Go to perform the origin TLS handshake and execute the request. Pooled connections reuse this established tunnel.

Implementation: Zero-Config SSO

```
import "github.com/sanjaynagpal/proxyauth"

// Discovers proxy from env vars and authenticates
// using the logged-in Windows user (SSO)
client := &http.Client{
    Transport: proxyauth.New(proxyauth.Config{}),
    Timeout:   30 * time.Second,
}
resp, err := client.Get("https://internal.example.com/api")
```

Zero configuration.
No passwords stored
or transmitted.

By default, proxyauth interfaces with the Windows OS to utilize the currently logged-in user's credentials seamlessly.

Explicit Control for Service Accounts

```
u, _ := url.Parse("http://proxy.corp.example:8080")
rt := proxyauth.New(proxyauth.Config{
    ProxyURL: u,
    Domain:   "CORP",
    Username: "svc-app",
    Password: os.Getenv("SVC_APP_PASSWORD"),
    // SPN: "HTTP/proxy.corp.example",
    // Defaults to HTTP/<proxy-host>
})
```

For automated CI runners or headless service accounts, developers can bypass SSO to inject explicit proxy URLs, domains, credentials, and custom Kerberos SPNs.

Additional Configs

Schemes

TLSClientConfig

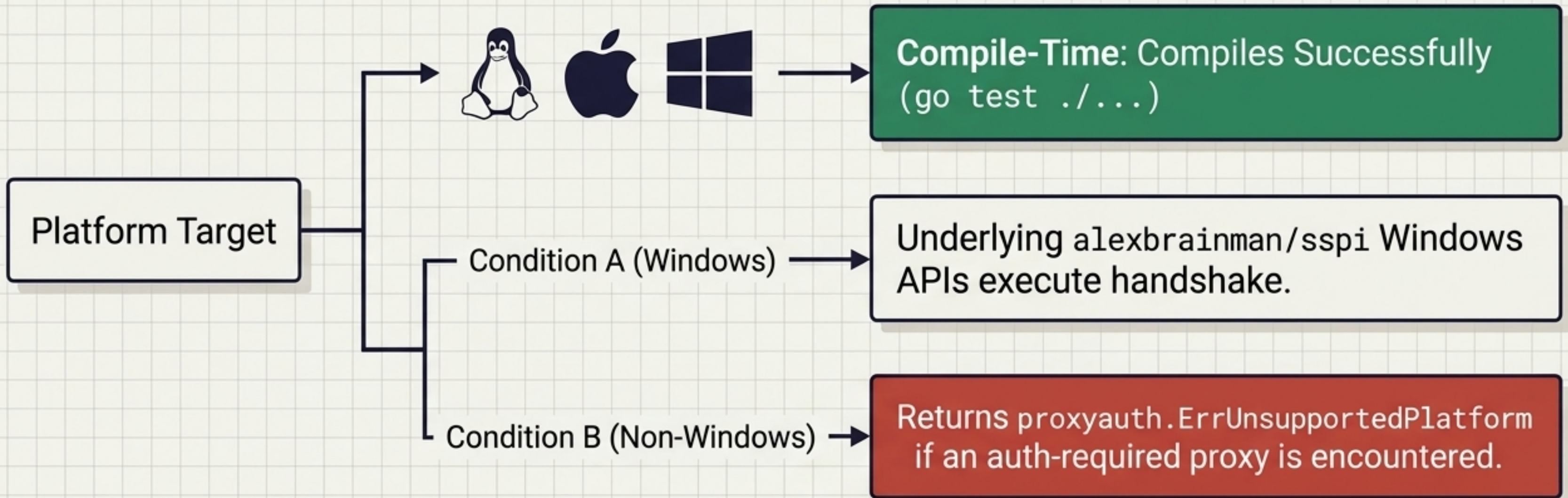
ProxyTLSClientConfig

DialContext

HandshakeTimeout

MaxLegs

Platform Support & CI Fallbacks



The core SSPI machinery relies on Windows-only APIs. However, the package compiles cleanly across all operating systems to ensure CI/CD testing pipelines and cross-platform builds never break.

Known Constraints & Edge Cases



Plaintext HTTP

Routed through a CONNECT tunnel to port 80. Proxies strictly blocking port 80 CONNECTs will reject these. HTTPS is unaffected.



Kerberos Limits

Requires a reachable KDC and accurate SPN (HTTP/<proxy-fqdn>)). SPNEGO automatically falls back to NTLM if Kerberos fails.



Header Parsing

Optimized for Negotiate, NTLM, and Basic. Not a full RFC 7235 parser (complex Digest challenges with comma-separated params may fail).

Designed primarily to solve the HTTPS / Windows SSPI problem, with specific structural tradeoffs.

Transparent Enterprise-Grade Connectivity

proxyauth provides robust corporate proxy traversal, integrating seamlessly with Windows SSPI. It isolates complex networking edge cases into a single wrapper, allowing Go developers to focus on application logic, not network topology.

